

TM4C123 LED Blinky — Stopwatch Extension

Implementation Report — EduARM4 Board (TM4C123GH6PM)

Objective

Extend the existing rate/colour LED blinky assignment with a stopwatch feature per the six assignment requirements: SysTick-based timekeeping, dual input (Serial + 4×4 keypad), a shared 7-segment display that switches between the stopwatch and the original RGB status readout, and interrupt-driven control for the onboard SW1/SW2 switches.

Implementation Summary

- Timekeeping: a SysTick interrupt ticks every 10 ms and drives both the stopwatch's elapsed-time counter (swCentis) and switch/keypad debounce timing — no busy-wait timing loops remain in the control path.
- Stopwatch actions: enable, disable, start, stop, pause, and resume are each a distinct explicit function (stopwatchEnable/Disable/Start/Stop/Pause/Resume), mirroring the LED side's RATE/COLOUR/PAUSE/RESUME style rather than a single toggle. Calling an action from an invalid state reports “already X” / “not valid” over UART instead of silently changing state.
- Serial console control: SWENABLE, SWDISABLE, SWSTART, SWSTOP, SWPAUSE, SWRESUME, and SWSTATUS, alongside the original RATE / COLOUR / PAUSE / RESUME / STATUS commands.
- Keypad control: row 0 (PE0) is held permanently low; columns PC4/PC5/PC6 map to keys ‘1’, ‘2’, ‘3’ for enable/disable, start/stop, and pause/resume, each firing a genuine falling-edge GPIO interrupt. The handler selects the correct explicit action for the current state, so one key still serves as an alternate for two actions, per the assignment brief, while sharing the same underlying functions as the UART commands.
- Display sharing (enable/disable): when the stopwatch is enabled the 7-segment module shows elapsed time as MM:SS; when disabled it reverts to the original rate/colour/run-state readout, via a single updateDisplay() dispatcher.
- Interrupt-driven SW1/SW2: onboard switches are handled through the GPIO Port F interrupt path rather than polling, preserving the original hold-to-increment and hold-both-to-pause behaviour.
- Vector table: SysTick_Handler and GPIOPortC_Handler are wired directly into the project's own startup file's vector table (entries 15 and 18), replacing an earlier RAM vector-table relocation workaround that is no longer needed.

Files Delivered

- main.c — application logic: LED blink/colour control, UART command parser, stopwatch state machine with explicit per-action functions, keypad and switch interrupt handlers, 7-segment display dispatch.
- startup_ccs_patched.c — the project's CCS/GNU startup file with SysTick and GPIO Port C vector table entries pointed at the application's handlers.

Verification Notes

Recommended checks: (1) confirm RATE/COLOUR/PAUSE/RESUME/STATUS still behave as in the base assignment; (2) SWENABLE then SWSTART and confirm the display switches to a running MM:SS count; (3) SWPAUSE freezes the count, SWRESUME continues it, and SWSTOP resets to 0; (4) each command sent from an invalid state (e.g. SWSTART while already running) reports a message rather than changing state; (5) keys ‘1’/‘2’/‘3’ on the keypad reproduce the same actions independently of the serial console.